

FleetFlow

Product Requirements

Document

Smart Logistics & Fleet Management Platform

Document Version: 1.0

Status: Active Development

Last Updated: 26 September 2026

A production-oriented PRD defining FleetFlow's product vision, users, capabilities, business rules, scope, acceptance criteria, and implementation roadmap.

1. Executive Summary

FleetFlow is a web-based fleet and logistics management platform designed to help organizations manage vehicles, requests, drivers, service history, maintenance workflows, notifications, and operational reporting from a centralized system.

The product is being developed with a Next.js and TypeScript frontend and a modular microservices backend. Authentication is handled by a dedicated authentication service, while fleet domains such as vehicles are separated into their own service boundaries.

The primary product objective is to provide a reliable role-based operational platform while demonstrating production-oriented software engineering practices such as API contracts, validation, RBAC, secure authentication, service boundaries, testing, documentation, and deployability.

2. Product Vision

Vision: Build a modern, secure, and extensible fleet management platform that gives every stakeholder the information and workflows needed to manage fleet operations efficiently.

Product principle: FleetFlow should reduce fragmented fleet administration by bringing vehicle data, requests, operational assignments, maintenance history, and reporting into one coherent platform.

3. Problem Statement

Fleet operations can become difficult to manage when vehicle information, service history, resource requests, driver assignments, and operational status are scattered across spreadsheets, messages, paper records, or disconnected applications.

FleetFlow addresses this problem by providing a structured digital workflow with role-based access, centralized vehicle information, controlled request lifecycles, historical records, and an architecture that can grow with the organization.

4. Product Goals

- Centralize fleet and vehicle information.
- Provide secure role-based access for Admin, Manager, Driver, and Customer users.
- Digitize vehicle request and approval workflows.
- Maintain reliable vehicle service and maintenance history.
- Provide managers with useful operational visibility.
- Create an extensible microservices foundation.
- Provide a responsive and professional web experience.
- Prepare the platform for notifications, analytics, automated testing, monitoring, and production deployment.

5. Non-Goals for the Initial Product

- Full GPS hardware integration is not required for the initial release.
- Advanced route optimization is not required for the initial release.
- Native mobile applications are outside the initial web-first scope.
- Complex financial/accounting functionality is outside the initial scope.
- External enterprise integrations should be added only after the core workflows are stable.

6. Target Users & Personas

Persona	Primary Need	Core Activities
Admin	Platform control	Users, roles, fleet oversight, configuration, reporting
Manager	Operational control	Vehicles, requests, assignments, service/history, analytics
Driver	Assigned work	Assigned vehicle/tasks, operational information, permitted updates
Customer	Request resources	Submit requests, monitor status, review request history

7. User Roles & Authorization

FleetFlow uses role-based access control. The frontend may adapt the interface based on the user's role, but authorization must always be enforced by backend services.

Role	Expected Access
Admin	Platform-wide administration and authorized fleet operations
Manager	Fleet operations, vehicles, requests, service/history and operational reporting
Driver	Driver-specific operational workflows and assigned resources
Customer	Vehicle/fleet requests and request tracking

8. Core Product Modules

Authentication: Registration, login, JWT authentication, protected resources, role-aware identity.

Dashboard: Role-aware summaries, quick actions, fleet information, recent activity and operational metrics.

Vehicle Management: Create, list, retrieve, update, deactivate and manage vehicle status.

Vehicle Requests: Create and review requests, track lifecycle, approve/reject where authorized, and support assignment workflows.

Vehicle Service & History: Store and display service records associated with vehicles.

Driver Management: Driver profiles, status, assignment and operational workflows.

Maintenance: Maintenance records, schedules, costs, status and reminders.

Notifications: Request, assignment, maintenance and system notifications with read/unread state.

Analytics & Reporting: Fleet utilization, status, requests, maintenance and operational reporting.

9. Key User Workflows

9.1 Registration and Login

- User opens the FleetFlow web application.
- User submits valid registration or login credentials.
- Auth service validates the request.
- Password authentication is performed where applicable.
- Auth service issues an authentication token.

- Frontend stores authenticated user state and role information.
- Protected API requests include authentication credentials.

9.2 Vehicle Management

- Authorized user opens vehicle management.
- User creates or selects a vehicle.
- Backend validates vehicle data.
- Vehicle service persists the data.
- The UI displays the updated fleet state.
- Vehicle status changes remain controlled by the backend.

9.3 Vehicle Request

- Authorized requester creates a vehicle request.
- Request is validated and persisted.
- Authorized manager reviews the request.
- Request status moves through controlled lifecycle states.
- Assignment can be introduced as part of the operational workflow.
- Requester can view the current status and history.

9.4 Vehicle Service / History

- Authorized user selects a vehicle.
- User records a service event.
- Service date and details are validated.
- Record is associated with the vehicle.
- Vehicle history displays chronological service information.
- Historical records remain available for future reporting and maintenance workflows.

10. Functional Requirements

ID	Requirement	Priority
FR-001	Users can register.	Must
FR-002	Users can authenticate securely.	Must
FR-003	System identifies user role.	Must
FR-004	Protected resources require authentication.	Must
FR-005	Authorized users can manage vehicles.	Must
FR-006	Users can view vehicle information.	Must
FR-007	Permitted users can submit vehicle requests.	Must
FR-008	Authorized users can review requests.	Must
FR-009	Vehicle service/history can be recorded.	Must
FR-010	Historical records remain associated with vehicles.	Must
FR-011	Unauthorized operations are rejected server-side.	Must

ID	Requirement	Priority
FR-012	Application supports responsive layouts.	Should
FR-013	Application supports light/dark themes.	Should
FR-014	Fleet analytics can be introduced.	Should
FR-015	Notifications can be introduced.	Should

11. Business Rules

- A user must be authenticated before accessing protected resources.
- A user's role determines which operations they may perform.
- Backend authorization is authoritative; frontend visibility is not a security mechanism.
- Deactivated vehicles should not be treated as normally available.
- Vehicle requests use controlled lifecycle states.
- Historical service records should be preserved rather than silently overwritten.
- Microservices should communicate through defined APIs/events instead of direct database access.
- Secrets must never be committed to source control.

12. Non-Functional Requirements

Area	Requirement
Security	Password hashing, JWT validation, RBAC, input validation, secret management and safe errors.
Performance	Efficient API/database queries, sensible pagination/filtering, and minimal unnecessary frontend requests.
Reliability	Graceful failures, consistent API errors, loading/empty/error states.
Scalability	Independent service boundaries, stateless services where practical, domain ownership.
Maintainability	TypeScript, clear service responsibilities, reusable components and documented contracts.
Usability	Responsive, accessible, consistent and role-aware interface.

13. Technical Product Constraints

- Frontend: Next.js 16.3.6 with React and TypeScript.
- Backend services: Fastify and TypeScript where applicable.
- Database: MongoDB.
- Authentication: JWT-based authentication.
- Validation: Zod-based validation where applicable.
- Frontend development server: port 3000.
- Authentication service development server: port 4001.
- Environment-specific configuration must be supplied through environment variables.

14. Success Criteria

The product should be considered successful when the core fleet workflow can be completed reliably from authentication through vehicle management, request handling, and service/history tracking.

- Users can authenticate without exposing credentials or secrets.
- Role-based restrictions work consistently at the backend.
- Authorized users can manage the fleet.
- Vehicle requests have a clear and traceable lifecycle.
- Vehicle service/history records are persistent and easy to review.
- The interface is usable across desktop and mobile layouts.
- Core APIs have predictable validation and error behavior.
- The system can be extended with additional services without redesigning the entire application.

15. Release Scope

Release	Scope
MVP	Authentication, roles, dashboard, vehicle management, basic vehicle requests, vehicle service/history.
Operations	Assignments, driver management, expanded request lifecycle, maintenance workflows.
Platform	Notifications, analytics, reporting, auditability and operational visibility.
Production	Automated tests, security hardening, monitoring, backups and deployment.

16. Product Roadmap

Phase	Focus	Status
1	Foundation, authentication, roles, dashboard	Implemented / active
2	Vehicle management and service/history	Implemented / active
3	Vehicle requests and assignment	In progress
4	Driver and fleet operations	Planned
5	Maintenance	Planned
6	Notifications	Planned
7	Analytics and reporting	Planned
8	Testing and production hardening	Planned
9	Deployment and monitoring	Planned

17. Definition of Done

- Frontend workflow is implemented.
- Backend endpoint/business logic exists where required.
- Request validation exists.
- Authorization exists.
- Data persistence is verified.
- Loading, empty and error states are handled.
- Responsive behavior is verified.
- API documentation is updated.

- Relevant tests are added as test infrastructure becomes available.
- The change is committed with a clear Git commit message.

18. Risks & Mitigations

Risk	Mitigation
Microservice complexity grows too quickly	Introduce services by domain and avoid unnecessary distributed infrastructure early.
Frontend/backend contracts drift	Maintain API documentation and validate request/response contracts.
Authorization implemented only in UI	Enforce RBAC on every protected backend operation.
Sensitive secrets committed	Use .env files locally, .env.example placeholders, secret rotation and Git hygiene.
Feature scope becomes too broad	Follow the roadmap and stabilize each core workflow before adding secondary features.
Insufficient test coverage	Add unit, integration and end-to-end testing before production release.

19. Future Opportunities

- GPS and real-time vehicle tracking
- Fuel and expense management
- Route planning and optimization
- Driver performance analytics
- Automated maintenance reminders
- Document and compliance management
- External logistics/ERP integrations
- Mobile applications
- Advanced operational dashboards

20. Product Documentation Ownership

The PRD defines product intent and scope. The engineering documents define implementation-level architecture, APIs, security practices, and development roadmap. When a major product decision changes, the PRD and the affected engineering documents should be updated together.

Related engineering documents: ARCHITECTURE.md, REQUIREMENTS.md, API.md, SECURITY.md, ROADMAP.md